

DAM - M08 – Aplicaciones móviles

Videojuegos

Txema Herrera

Práctica

Práctica 1

Programar en Android un juego en donde aparezcan naves de forma aleatoria desde la izquierda de la pantalla hacia la derecha. A la derecha de la pantalla se visualizará mi nave, la cual sólo se puede mover de arriba abajo, no se puede mover de derecha a izquierda. El control de mi nave se ha de poder realizar desde teclado y desde ratón.

A medida que pase el tiempo, la velocidad de las naves de ataque aumentará. – 1 punto

A la que se detecte una única colisión ha de salir la pantalla de Game Over. – 1 punto

El juego debe tener sonidos diferentes. El sonido mientras se juega y el sonido al recibir una colisión. – 1 punto

La primera pantalla que aparezca ha de ser aquella en donde se introduzca el nombre y dos niveles de dificultad del juego. El nombre tiene que aparecer en la segunda pantalla. 1 punto

Mi nave ha de poder disparar. – 1 punto

El nivel de dificultad tendrá una relación directa con el número de naves atacantes. – 1 punto

Las naves atacantes sólo pueden disparar de forma aleatoria en el segundo nivel – 1 punto

Implementación aspectos a mejorar – 2 puntos

Video explicativo en donde se muestra la funcionalidad y el lugar en donde está implementado – 3 puntos

Inglés – 1 punto

La entrega de la actividad estará formada por

1. *El proyecto que hayas creado en **Android Studio** como solución de la actividad, comprimido en formato **.zip**. No es necesario entregar todo el proyecto, sólo has de entregar el contenido de la carpeta “ app\src\main” del proyecto*
2. *El **apk** que lo encontrarás en el directorio “ app\build\outputs\apk\debug” del proyecto*
3. *Personalización de las **variables con tus iniciales** al final. Si una variable es el DNI, la variable se deberá llamar DNI_JMH (donde JMH son mis iniciales)*
4. *Un **video** en inglés en donde se vean claramente los “test-cases”, es decir, las ejecuciones con todas las comprobaciones del correcto funcionamiento. Si el video no te cabe en la entrega, súbelo al OneDrive (o equivalente) y pásame el enlace, recuerda añadir los permisos correspondientes*

Teoría

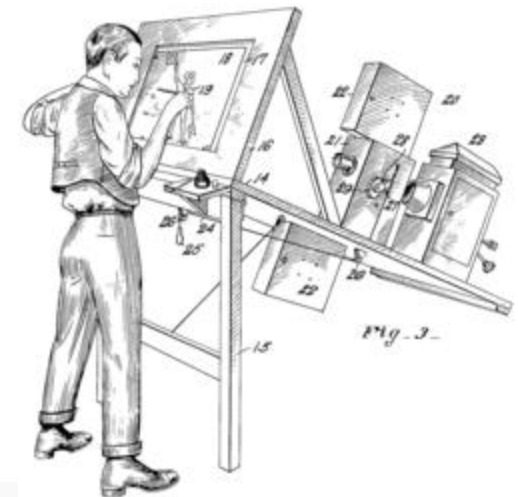
Contenido de la clase

- Conceptos teóricos
 - Definición y Tipos de Animación
 - Sprites / Textura
 - Renderizar
 - Colisiones
 - Sonidos
 - Tipos de vídeo juegos
 - Diseño de videojuego
 - Entradas de usuario
 - Dibujar elementos básicos
- Ejemplos prácticos

Definición y Tipos de Animación

La **animación** se realiza a partir de un solo gráfico, sobre el que se aplican transformaciones para conseguir un efecto realista. 24 fotogramas por segundo se considera animación

- **Stop-motion o fotograma.** Se capturan los movimientos de objetos inanimados realizados manualmente con una cámara fotográfica y después se unen y editan en una computadora
 - Cut-out: por recortes: recortes de personajes que movemos foto a foto
 - Claymation o plastinación: material moldeable que movemos y hacemos foto
 - Pixilación: los objetos son personas o elementos reales
 - Go motion: al hacer la foto se mueve el objeto y sale borrosa pero mejora la sensación de movimiento. Se usó en “El imperio contraataca.”
- **Rotoscopia:** Se copian los fotogramas de una filmación real y se sustituyen con dibujos creados digitalmente a partir de la imagen real. Esto permite agregar filtros, colores, efectos y estilos para transmitir un concepto estético. Un ejemplo son los “sables laser” de “star wars”
- **Animación digital**
 - 2D
 - 3D



Un sprite es la combinación de una imagen gráfica y su información geométrica.

La parte gráfica de un sprite viene determinada por una **textura**, una imagen **descomprimida y guardada en la memoria del procesador gráfico**.

Mediante el proceso de mapeo de texturas se calcula en qué lugar de la pantalla y con qué forma se va dibujar la textura.

Al Sprite le puedes aplicar movimiento, colisiones, ...; a la textura no.

Renderizar

Proceso de generar **digitalmente** imagen fotorrealista, o no, a partir de un modelo 2D o 3D.

Es el proceso más costoso de realizar en la animación digital ya que requiere de un alto nivel de procesamiento gráfico.

Colisiones

Las técnicas de detección de colisiones en los videojuegos se utilizan para controlar el movimiento de los sprites.

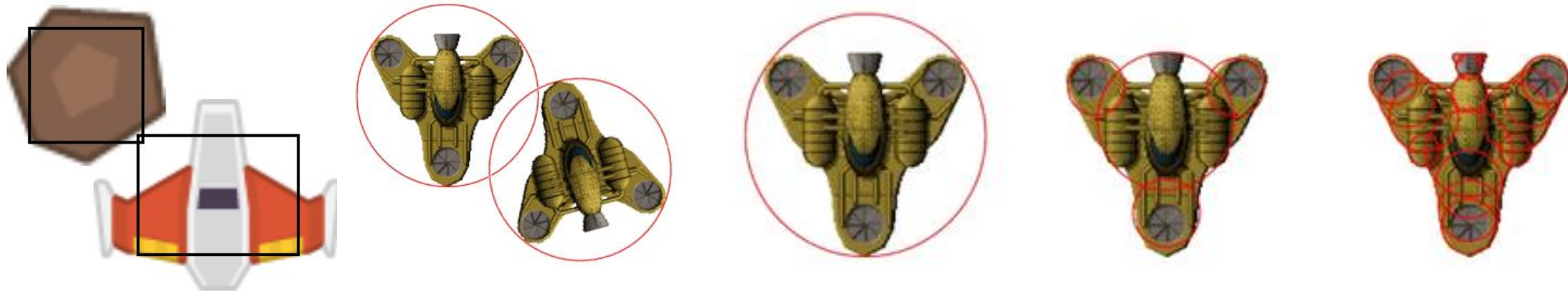
Mediante estas técnicas podemos:

- Evitar que los sprites se salgan de los límites de la pantalla, es decir, detectar cuando los sprites alcanzan los límites marcados.
- Evitar que los sprites atraviesen obstáculos u otros sprites que son considerados como sólidos (por ejemplo, una pared, una nave, un objeto en una habitación, ...).
- Detectar el momento en el que los sprites entran en contacto con objetos del juego.
- Simular el comportamiento de proyectiles; en ese caso, se aplicará la lógica del juego (producir una explosión, descontar una vida, ...).

Colisiones - Detección geométrica

Todo dependerá del tipo de juego, pero en cualquier caso la simplificación de la geometría del sprite es la clave para poder detectar las colisiones.

Una técnica muy sencilla consiste en envolver los sprites en figuras geométricas simples para de esa forma detectar si colisionan o no.

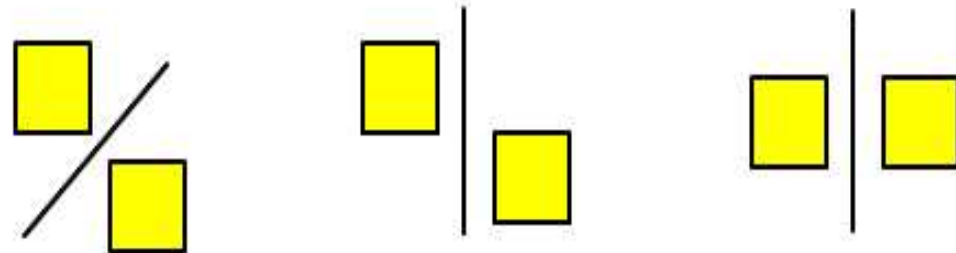


Colisiones - Detección mediante proyecciones

La técnica de proyección para detectar el solapamiento de dos sprites consiste en proyectar geométricamente los objetos sobre un plano.

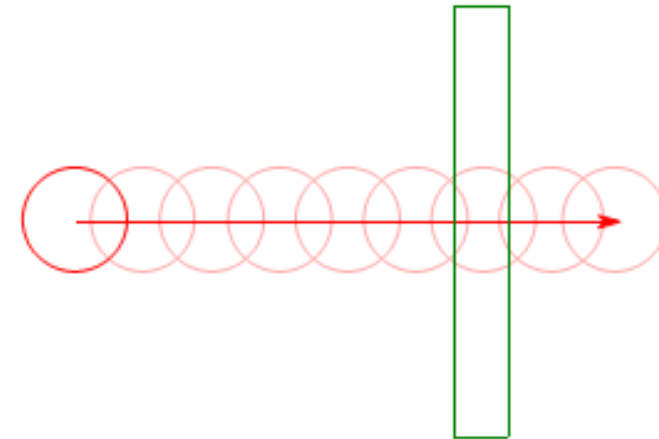
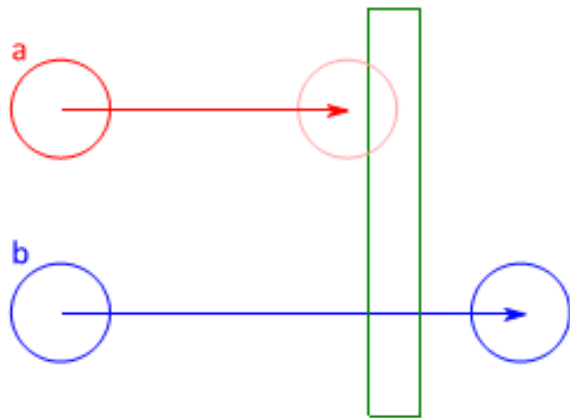
El Teorema de Separación de Ejes establece que si tenemos dos figuras convexas y encontramos un eje en el cual la proyección de las dos figuras no se solapa, entonces los dos sprites no han colisionado.

Esta técnica reduce la complejidad a un problema de una dimensión y es válida para la mayoría de los videojuegos, acelerando de esta manera los cálculos.



Colisiones - Otras consideraciones

La frecuencia de renderización puede influir en la detección de colisiones.



Librerías de sonidos

Los motores de juegos simplifican la tarea de realizar sonidos en los videojuegos proporcionando librerías de sonidos listos para ser utilizados.

- Por un lado, los sonidos melódicos se utilizan como **melodía de fondo** en muchos videojuegos.
- Por otro lado, los **efectos sonoros están asociados a cada uno de los elementos** del juego.

Además de las librerías de sonidos, algunos motores de juegos incluyen también **editores de sonido y herramientas de grabación** que permiten manipular, grabar y reproducir todo tipo de sonidos para ser utilizados en el videojuego, desde disparos y explosiones hasta conversaciones de los personajes.

Tipos de vídeo juegos

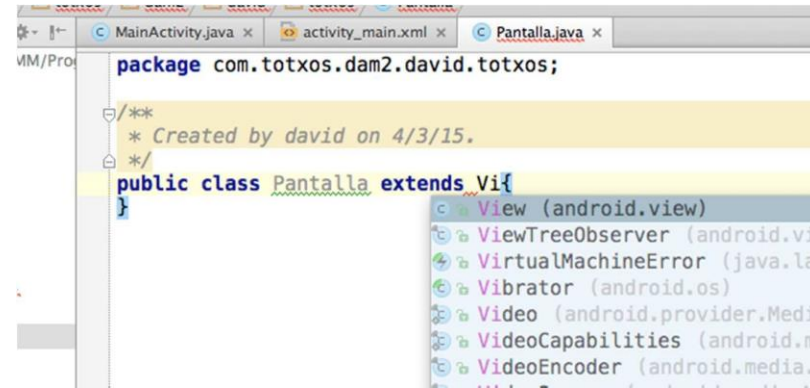
Se pueden combinar los dos tipos de clasificación

	Estáticos	Dinámico
2D	Buscaminas Apalabrados Solitarios de cartas	Tetris Arkanoid Galaxians
3D	Ajedrez 3D juegos tipo puzle tridimensionales	Quake World of Warcraft

(aunque los juegos estáticos 3D no son demasiado frecuentes)

¿Cómo diseñar un vídeo juego?

- Crear un nuevo proyecto con el asistente.
- Añadimos una nueva clase Java que herede de **View** . Aquí es donde realizaremos el **render**
- En el onDraw es donde dibujamos la escena



```
package com.totxos.dam2.david.totxos;

/**
 * Created by david on 4/3/15.
 */
public class Pantalla extends View {
```

- En el layout colocamos un elemento de tipo View
- Puede ser que necesitemos realizar un **Rebuild**

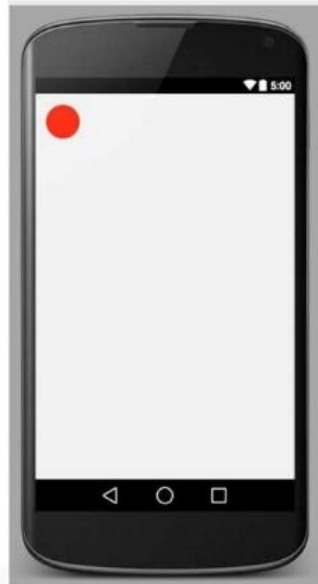
```
@Override
protected void onDraw(Canvas canvas) {
    super.onDraw(canvas);

    Paint p=new Paint();
    p.setColor(Color.RED);
    canvas.drawOval(new RectF(0,0,100,100),p);
}
```

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
    tools:context=".MainActivity">

    <view
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        class="com.totxos.dam2.david.totxos.Pantalla"
        android:id="@+id/Pantalla"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true"/>

</RelativeLayout>
```



Entradas del usuario

En los dispositivos Android existen muchos sensores que sirven de entrada de acciones para un videojuego. Entre ellos podemos encontrar:

- Pantalla táctil
- Micrófono
- Acelerómetro: sensor de movimiento capaz de saber la orientación del teléfono
- Barómetro: mide presión atmosférica. Ayuda a saber la altitud
- Giroscopio: vinculado al acelerómetro. Ayuda a simular el móvil como volante de un coche
- GPS
- Podómetro: medir los pasos
- Magnetómetro: mide campo magnético
- NFC: para pagos contactless
- Sensores biométricos
-

En el [siguiente](#) enlace podrás encontrar información de cómo usarlo en Android St

▼ Sensores

Descripción general

Información general de sensores

Sensores de movimiento

Sensores de posición

Sensores de entorno

Dimensiones GNSS sin procesar

Entradas del usuario

Un ejemplo de código para capturar pulsaciones del jugador en la pantalla y actuar en consecuencia se puede implementar sobrescribiendo el método `onTouchEvent()`:

```
@Override
public boolean onTouchEvent(MotionEvent event) {
    if (event.getAction() == MotionEvent.ACTION_DOWN) {
        float posicionY= event.getY();
        if (posicionY> escalon) {
            escalon = escalon + 1;}
        else {
            escalon = escalon - 1;}
    }
    invalidate(); // llamada a onDraw() para redibujar vista
    return true;
}
```

La lógica del juego está contenida en el bucle infinito del juego, que se repite muchas veces por segundo, y es la parte de código responsable de actualizar el estado de los elementos del juego

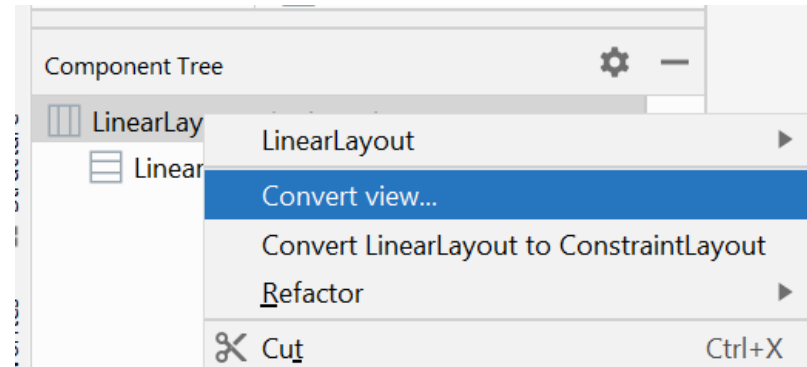
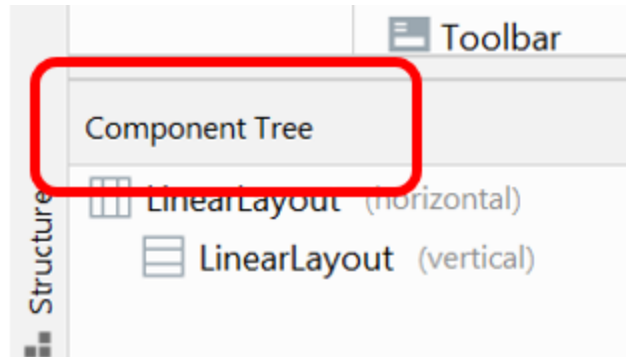
1. Comprobar los eventos del usuario: si el jugador ha pulsado sobre la pantalla, si ha oscilado el dispositivo, ...
2. Actualizar las posiciones de los elementos del juego: el personaje principal, los enemigos u otros integrantes, los ítems que hay que recoger,
3. Detectar colisiones entre elementos; por ejemplo, el personaje contra una pared que no puede atravesar o bien recoger un objeto.
4. Dibujar de nuevo la pantalla en función de todo lo anterior (movimientos, saltos, heridas, explosiones, ...)
5. Añadir un pequeño retardo o pausa para acomodar la velocidad de la acción.

Dibujar elementos básicos

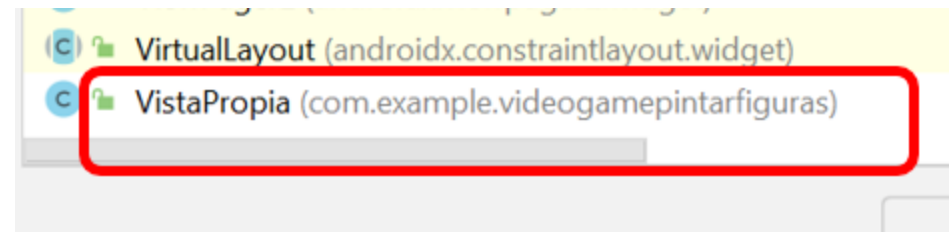
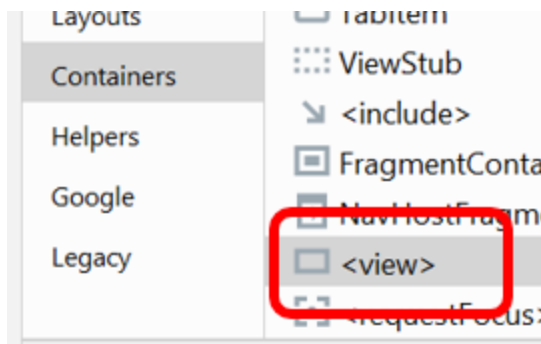
Lo primero que tenemos que realizar es crear una clase que herede de View

```
import android.content.Context;
import android.view.View;
public class VistaPropia extends View {
    public VistaPropia(Context context) {
        super(context);
    }
}
```

Borramos el TextView y cambiamos el Layout de tipo constraint a LinarLayout



Al arrastrar el elemento de tipo View nos deja escoger con qué java lo queremos asociar



Dibujar elementos básicos

Si lo ejecutamos tal cual nos dará el siguiente error

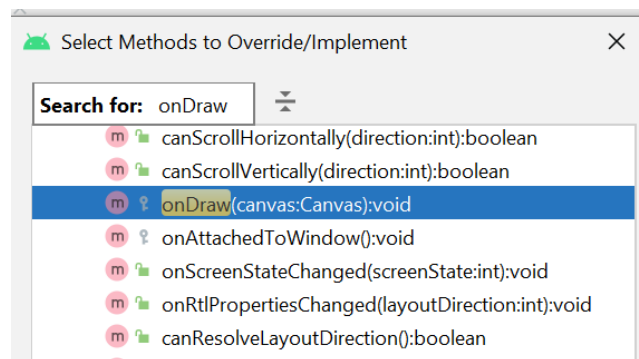
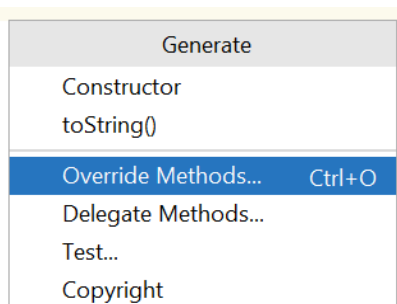
Error inflating class com.example.videogamepintarfiguras.VistaPropia

El motivo es que no hemos definido el constructor necesario para pintarlo. Lo añadimos y ya funciona [public](#)

```
VistaPropia(Context context, AttributeSet attrs) {  
    super(context, attrs);  
}
```

El siguiente paso es definir qué y cómo lo queremos pintar. Para ello creamos el onDraw

Apretamos el Alt+Insert y escogemos el elemento a sobrescribir

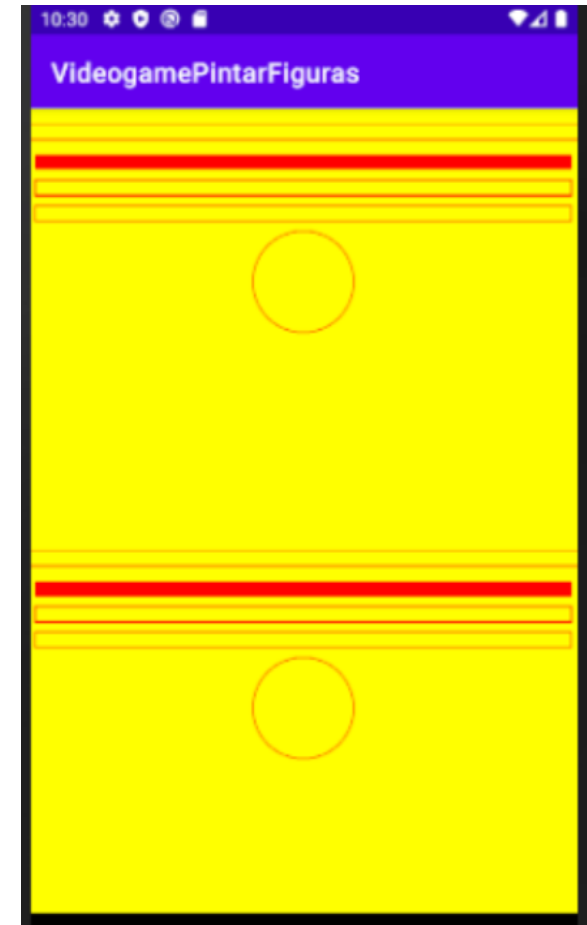


Dibujar elementos básicos

```
protected void onDraw(Canvas canvas) {  
    //Pintar fondo amarillo de dos diferentes formas  
    canvas.drawRGB(255, 255, 0);  
    canvas.drawColor(Color. YELLOW);  
    //Cogemos el ancho para no pintar una línea infinita  
    int ancho = canvas.getWidth();  
    //pincel1 es la variable usada para pintar  
    Paint pincel1 = new Paint();  
    // pintamos una línea de color rojo  
    pincel1.setARGB(255, 255, 0, 0);  
    int x=30;  
    canvas.drawLine(0, x, ancho, x, pincel1);  
    // la pintamos más gruesa  
    x=x+30;  
    pincel1.setStrokeWidth(4);  
    canvas.drawLine(0, x, ancho, x, pincel1);  
    //Pintamos rectángulos  
    x=x+30;  
    pincel1.setARGB(255,255,0,0);  
    canvas.drawRect(10,x,ancho-10,x+30,pincel1);  
    x=x+50;  
    pincel1.setStyle(Paint.Style. STROKE);  
    canvas.drawRect(10,x,ancho-10,x+30,pincel1);  
    x=x+50;  
    pincel1.setStrokeWidth(3);  
    canvas.drawRect(10,x,ancho-10,x+30,pincel1);  
    //Pintamos círculos  
    x=x+150;  
    canvas.drawCircle(ancho/2, x, 100, pincel1);  
}
```

Ya que hemos definido en el Layout dos elementos (un LinearLayout y un View), la llamada para que lo pinte es diferente.

```
LinearLayout layout1 =  
    findViewById(R.id. layoutPropio);  
VistaPropia vistaPropia = new VistaPropia(this);  
layout1.addView(vistaPropia);
```



Ejemplos proporcionados

El material lo puedes encontrar en el [siguiente](#) enlace

-  mainBuscaminas.zip
-  mainColisionesPuntuacion.zip
-  mainPintar10Circulos.zip
-  mainPintarLineasRecuadroCirculo.zip
-  mainVideoGameCaidaBola.zip
-  mainVideoGamiBasicMoverBola.zip

Buscaminas (videogamebuscaminasjava)

- Clase “Casilla” utilizada para almacenar las casillas del buscaminas, en donde estarán las minas o estarán vacías.
- Clase “Tablero” que hereda de “View”. Es la clase usada para pintar la tabla con las minas. En el ejemplo es una tabla de 8x8
 - Dentro de esta clase se define el onDraw. Función usada para pintar cualquier actualización de lo que suceda
- Tenemos un Layout en donde se ponen dos elementos
 - Botón usado para reiniciar la partida
 - LinearLayout, que será usado para pintar el tablero de las minas

- El MainActivity.java hereda de **View OnTouchListener** en dónde. encontramos
 - onCreate
 - llamamos al layout principal “activity_main”
 - definimos el tablero que es de tipo View
 - definimos el listener onTouch como this, es decir, el contexto actual
 - al LinearLayout le añadimos la vista definida, es decir, el tablero.
 - Pintamos la bombas de forma aleatoria
 - Pintamos el perímetro de las bombas
 - Escondemos los elementos para que no se venal
 - onTouch. Evento que podemos definir ya que el MainActivity hereda de View.OnTouchListener. En el definimos las acciones a realizar cada vez que el usuario realice un “click” sobre la pantalla.
 - Aquí es donde averiguamos la posición de donde a realizado el click el usuario
 - También llamamos al **invalidate**, que se encarga de volver a cargar el tablero.
 - Definimos la clase “Tablero” anteriormente comentada.
 - Funciones necesarias para la lógica del juego: Reinicar, DisponerBomba, Gano, ContarBombaPerimetro,

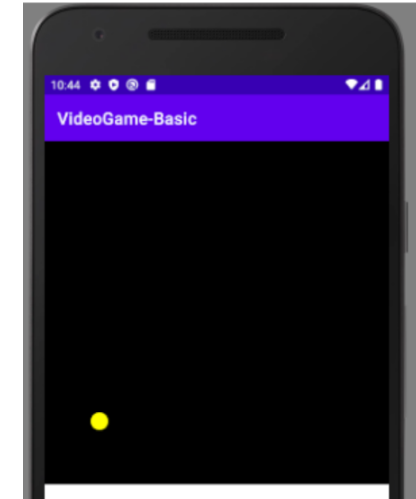
- Clase “Juego” que hereda de “View”. Es la clase usada para pintar el juego en donde caen las bolas. Dentro de esta clase se define
 - onTouchEvent. Encargada de detectar las acciones del usuario para volver a pintar llamando al invalidate (este comando llama al onDraw)
 - el onDraw. Función usada para pintar cualquier actualización de lo que suceda
- Tenemos un Layout en donde ponemos un solo elemento de tipo “View”
- El MainActivity. en donde. Encontramos en el onCreate
 - llamamos al layout principal “activity_main”
 - definimos variable “juego” de tipo “Juego” (es decir, de tipo “View”) para pintar el juego por pantalla
 - creamos un listener (observador) que se ejecutará solo que el layout se acabe de pintar
 - definimos un Handler para usarlo en un timer. Será el encargado de realizar la **renderización** para simular el movimiento.

Si realizamos una comparativa con el código del buscaminas encontramos las siguientes diferencias:

- El buscaminas no necesita renderización y por lo tanto no tiene timer.
- El buscaminas define el onTouchEvent en el MainActivity y en este ejemplo se define en la clase que hereda del View

Mover una bola

```
@Override
public boolean onTouchEvent(MotionEvent event) {
    int eventAction = event.getAction();
    // you may need the x/y location
    int x = (int)event.getX();
    int y = (int)event.getY();
    switch (eventAction) {
        case MotionEvent.ACTION_DOWN:
            break;
        case MotionEvent.ACTION_UP:
            break;
        case MotionEvent.ACTION_MOVE:
            posY=y;
            posX=x;
            radio=25;
            // invalidate llama al onDraw y vuelve a pintar la bola
            this.invalidate();
    }
    return true;
}
```



Al realizar un click sin movimiento la pelota no se modifica.

Al realizar un click con movimiento la pelota se mueve y se hace más pequeña

Bolas en caida

```
//Ejecutamos cada 20 milisegundos
Timer timer = new Timer();
timer.schedule(new TimerTask() {
    @Override
    public void run() {
        handler.post(new Runnable() {
            public void run() {
                //Cada x segundos movemos la moneda 10dp
                juego.posMonedaY+=10;
                //refreca la pantalla y llama al draw
                juego.invalidate();
            }
        });
    }
}, 0, 20);

@SuppressLint("DrawAllocation")
@Override
protected void onDraw(Canvas canvas) {
    super.onDraw(canvas);
    //Definimos los objetos a pintar
    .....
    //Pintamos moneda
    if (posMonedaY>alto) {
        posMonedaY=50;
        posMonedaX= random.nextInt(ancho);
    }
    rectMoneda = new RectF((posMonedaX-radio), (posMonedaY-
radio), (posMonedaX+radio), (posMonedaY+radio));
    canvas.drawOval(rectMoneda, moneda);
}
```

Colisiones y puntuación

En el onDraw

```
// Calculo intersección  
if (RectF.intersects(rectCesta,rectMoneda)) {  
    puntuacion += 1;  
    posMonedaY=50;  
    posMonedaX= random.nextInt(ancho);  
}
```

Añadir sonido

En el constructor

```
public Juego(Context context, AttributeSet attrs) {  
    super(context, attrs);  
    gameloop = MediaPlayer.create(context, R.raw.gameloop);  
    gameloop.start();  
    //mantiene el loop del soundtrack  
    gameloop.setOnCompletionListener(new MediaPlayer.OnCompletionListener() {  
        @Override  
        public void onCompletion(MediaPlayer mp) {  
            gameloop.start();  
        }  
    });  
}
```

Añadir enemigo

```
//Ejecutamos cada 20 milisegundos
Timer timer = new Timer();
timer.schedule(new TimerTask() {
    @Override
    public void run() {
        handler.post(new Runnable() {
            public void run() {
                //Cada x segundos movemos la moneda
                juego.posMonedaY+=10;
                juego.posMonedaFalsaY+=10;
                //refreca la pantalla y llama al draw
                juego.invalidate();
            }
        });
    }
}, 0, 20);

// Calculo intersección
if (RectF.intersects(rectCesta,rectMonedaFalsa)) {
    puntuacion -= 5;
    posMonedaFalsaY=50;
    posMonedaFalsaX= random.nextInt(ancho);
}
```

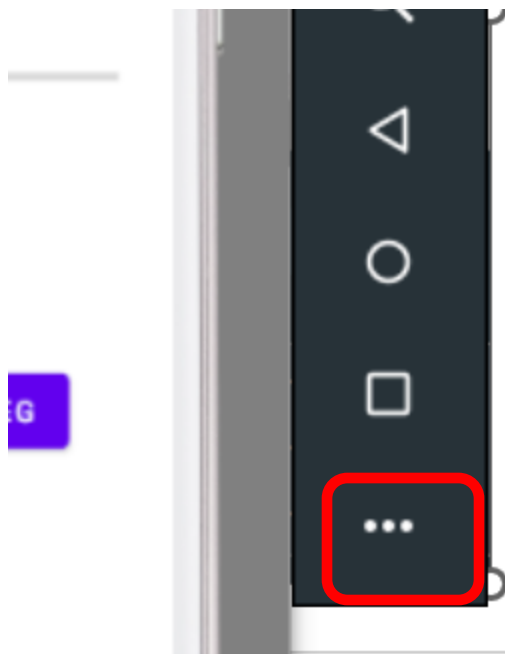

OJO con el tamaño de la imagen, ralentiza mucho el juego

- Añado la imagen en el directorio drawable.
- Creo variable Bitmap
- Uso el “drawBitmap”

```
Bitmap bitmapHucha = BitmapFactory.decodeResource(getResources(),  
R.drawable.hucha);
```

```
//canvas.drawOval(rectCesta, cesta);  
canvas.drawBitmap(bitmapHucha, null, rectCesta, cesta);
```

Acelerómetro en emulador Android Studio



Location
Displays
Cellular
Battery
Camera
Phone
Directional pad
Microphone
Fingerprint
Virtual sensors
Bug report
Snapshots
Record and Playback
Settings
Help

Device Pose Additional sensors

Rotate Move

Z-Rot -180 180 1.0

X-Rot -180 180 1.0

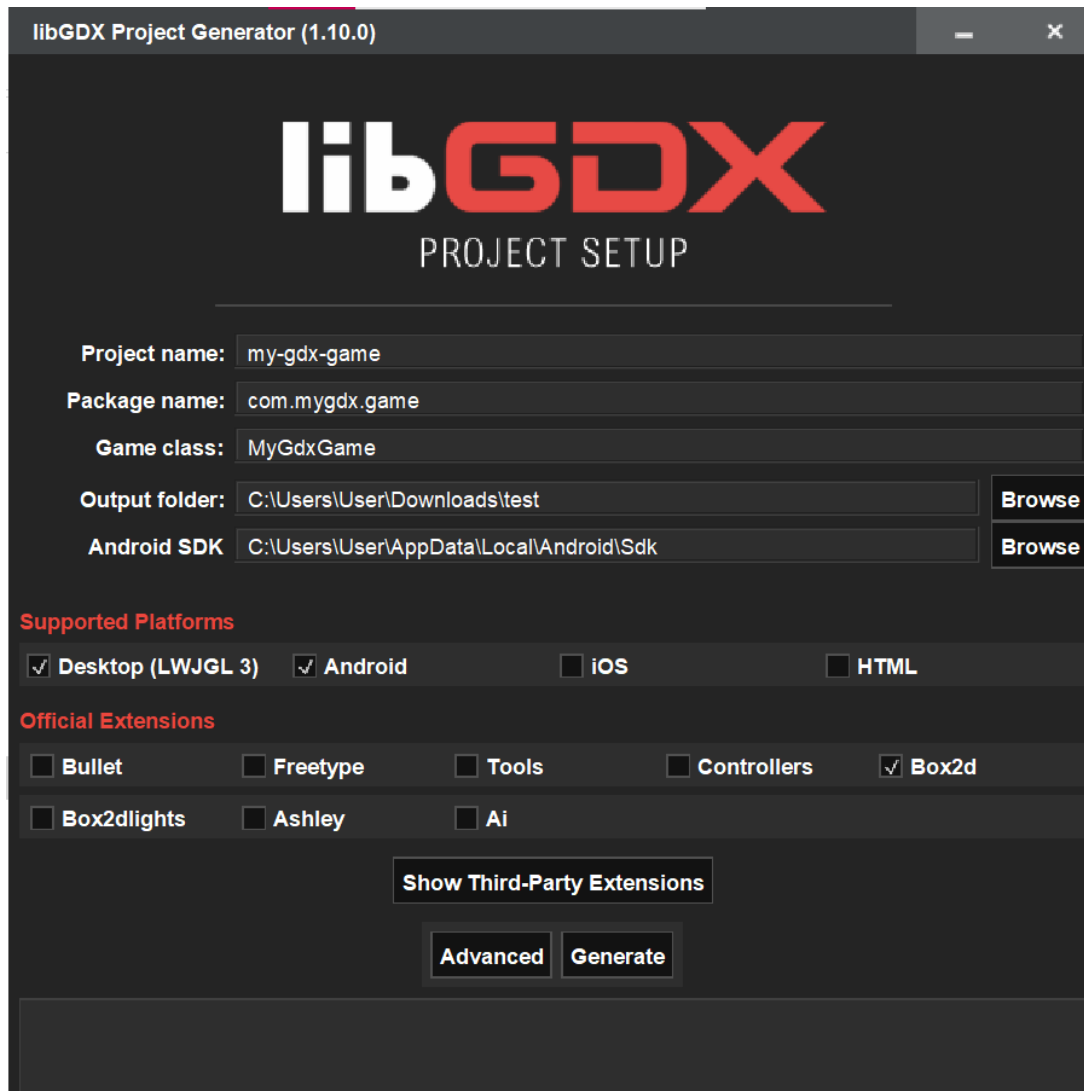
Y-Rot -180 180 7.5

Rotation

Sensor values

Accelerometer (m/s ²):	0.19	9.81	-0.17
Gyroscope (rad/s):	0.00	0.00	0.00
Magnetometer (μT):	6.44	4.92	-48.08

LibGDX



libGDX Project Generator (1.10.0)

libGDX
PROJECT SETUP

Project name: my-gdx-game

Package name: com.mygdx.game

Game class: MyGdxGame

Output folder: C:\Users\User\Downloads\test **Browse**

Android SDK: C:\Users\User\AppData\Local\Android\Sdk **Browse**

Supported Platforms

☒ Desktop (LWJGL 3) ☒ Android ☐ iOS ☐ HTML

Official Extensions

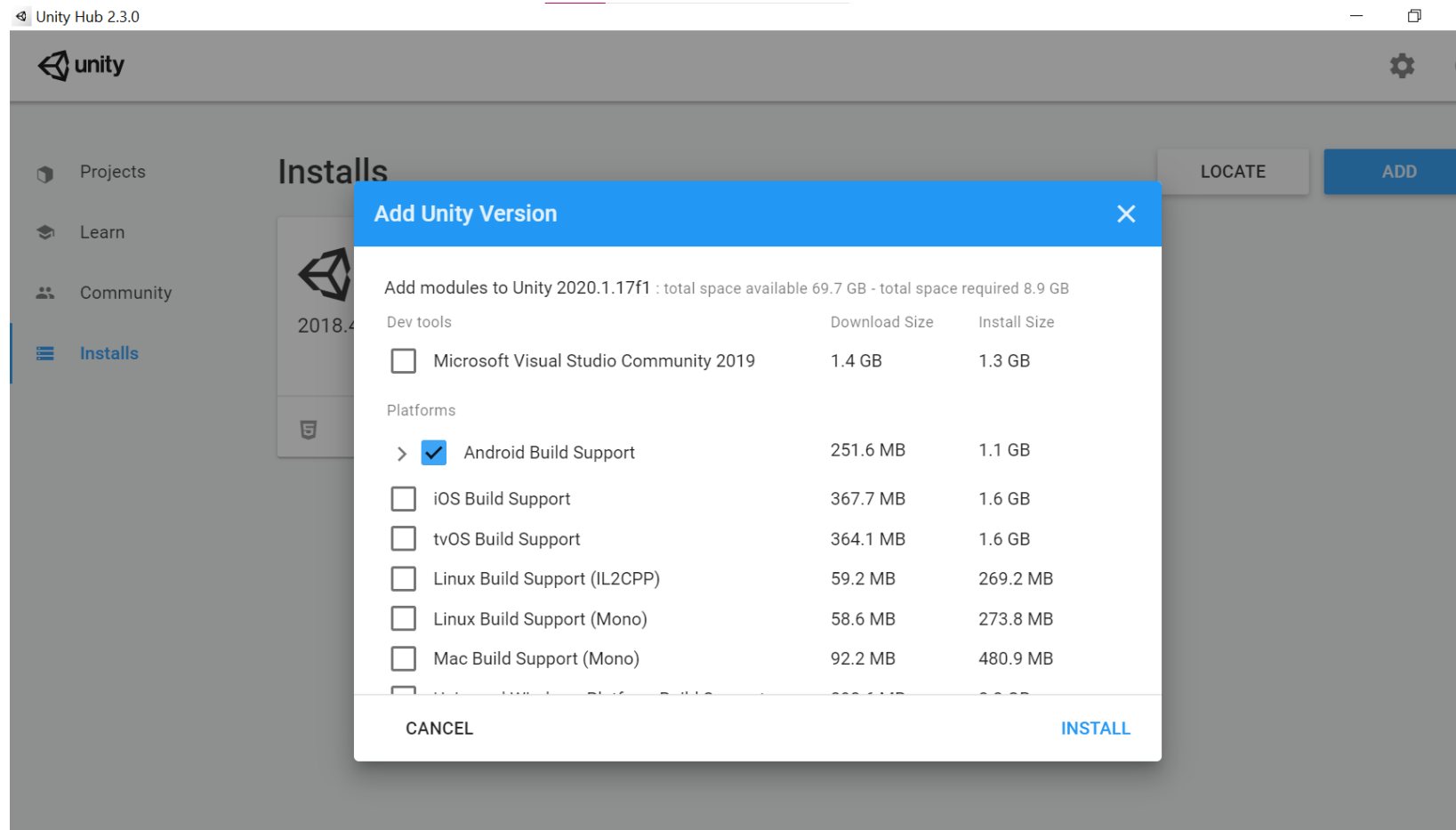
☐ Bullet ☐ Freetype ☐ Tools ☐ Controllers ☒ Box2d

☐ Box2dlights ☐ Ashley ☐ Ai

Show Third-Party Extensions

Advanced **Generate**

Documento en la misma carpeta
que las presentaciones



Documento en la misma carpeta que las presentaciones

